TRƯỜNG ĐẠI HỌC PHENIKAA

KHOA HỆ THỐNG THÔNG TIN

BÁO CÁO BÀI TẬP LỚN

Mô hình hóa và kiểm chứng hình thức với mô hình Kripke

Hệ thống nghiên cứu: Smart Lock

CSE703093 - An toàn phần mềm

Giảng viên hướng dẫn: ThS. Vũ Quang Dũng

Lớp: [NHÓM ĐIỀN] | Học kỳ, năm học: [NHÓM ĐIỀN]

Thành viên	Họ và tên	MSSV
Người 1	[NHÓM ĐIỀN]	[NHÓM ĐIỀN]
Người 2	[NHÓM ĐIỀN]	[NHÓM ĐIỀN]
Người 3	[NHÓM ĐIỀN]	[NHÓM ĐIỀN]
Người 4	[NHÓM ĐIỀN]	[NHÓM ĐIỀN]

Chủ đề 1: Mô hình hóa và kiểm chứng hình thức (Kripke + BMC)

Ngày tổng hợp bằng chứng: 15/10/2026

MỤC LỤC

Chương 1. Giới thiệu đề tài 3

Chương 2. Đặc tả hệ thống và mô hình Kripke 4

Chương 3. Đặc tả thuộc tính CTL 8

Chương 4. Kiểm chứng bằng BMC và Z3 11

Chương 5. Sinh mã và kết quả thực nghiệm 15

Chương 6. Thảo luận và hạn chế 19

Chương 7. Kết luận và tài liệu tham khảo 20

Phụ lục A. Kripke tổng thể v0 21

Phụ lục B. Kripke tổng thể v1 22

Phụ lục C. Admin Recovery 23

Phụ lục D. Lệnh chạy và chỉ mục bằng chứng 24

Mục lục được tạo tự động theo số trang PDF; số trang tính từ bìa. Các mục phụ lục nằm sau phần nội dung chính.

Chương 1. Giới thiệu đề tài

1.1. Bối cảnh và bài toán

Một lỗi kiểm tra quyền có thể làm phần mềm thực hiện thao tác nhạy cảm ngay cả khi người dùng chưa xác thực. Một quy trình phục hồi có thể luôn có lựa chọn hoàn tất nhưng vẫn cho phép chờ lặp vô hạn. Hai tình huống này đòi hỏi cách lập luận khác nhau: loại trừ trạng thái xấu và bảo đảm tiến triển trên mọi nhánh thực thi.

Đề tài **Mô hình hóa và kiểm chứng hình thức với mô hình Kripke** chọn Smart Lock làm chương trình mục tiêu. Đây là hệ thống đủ nhỏ để đọc toàn bộ logic, nhưng có quan hệ giữa xác thực, khóa tài khoản, cửa, báo động và phục hồi. Lựa chọn phù hợp chủ đề 1 trong hướng dẫn giảng viên [1, tr. 9-10]. Chương 1 của học phần được áp dụng bằng mã hóa trạng thái và BFS thật; Chương 3 được áp dụng bằng CTL, BMC và Z3, thay vì chỉ mô tả lý thuyết. Không mở rộng sang các chủ đề kiểm toán bộ nhớ hoặc fuzzing.

1.2. Mục tiêu, phạm vi và công cụ

Mục tiêu là xây dựng một mô hình tích hợp cho mỗi phiên bản, đặc tả đúng năm thuộc tính CTL, phát hiện hai lỗi cố ý của v0, xác nhận bản sửa v1, và sinh simulator Python bảo toàn các transition. Mọi kết luận đều gắn với JSON, CSV hoặc log thực tế trong gói nộp.

- F1 Authentication: tiếp nhận kết quả mật khẩu đúng/sai.
- F2 Account Lockout: khóa sau đúng ba lần thử sai liên tiếp.
- F3 Door Control: mở/đóng cửa gắn với phiên xác thực.
- F4 Alarm: tamper kích hoạt báo động và đóng cửa.
- F5 Admin Recovery: yêu cầu quyền quản trị và hoàn tất reset về idle.

Python biểu diễn logic; lớp KripkeStructure của softsec-toolkit lưu S/S0/R/L; Z3 giải ràng buộc BMC; pytest kiểm thử; Graphviz tạo hình. Không có cơ sở dữ liệu, mạng, phần cứng khóa hoặc mật khẩu thật. Kết quả đầu vào xác thực được trừu tượng hóa thành event hoặc bool.

1.3. Quy trình và bằng chứng

Luồng thực hiện là: guard/action → BFS → Kripke G2 → CTL/BMC G3 → mã sinh và sơ đồ G4 → báo cáo G5. Cả CTL, SMT và generator dùng cùng mô hình G2. G5 chỉ tổng hợp và chạy lại tests; không sửa logic đã nghiệm thu. Hai snapshot có 21/18 state reachable; G3 có 44 query Z3; G4 đối chiếu 702 bộ input và toàn bộ 115 transition. Kết quả chi tiết nằm ở Chương 5.

Quy mô mã lõi: model.py 140 dòng/4 hàm-method; kripke.py 256/11; ctl.py 179/8; bmc.py 126/4; artifacts.py 371/8. Đây là đếm dòng vật lý và định nghĩa hàm, gồm method, không phải chỉ số độ phức tạp hay benchmark; nguồn đo được lưu ở results/g5/evidence_audit.json.

Chương 2. Đặc tả hệ thống và mô hình Kripke

2.1. Sáu biến và trạng thái khởi tạo

Một state là bộ sáu giá trị, theo thứ tự thực tế trong `smartlock/model.py::State`. Dataclass frozen cho phép hash và so sánh toàn bộ giá trị khi BFS phát hiện trùng. ID s0, s1 chỉ phục vụ hiển thị.

Biến	Miền	Ý nghĩa
authenticated	bool	Phiên người dùng đã xác thực; xóa khi đóng cửa, tamper, reset
failed_attempts	int 0..3	Số lần thử sai kể từ lần đúng hoặc reset hoàn tất gần nhất
door_open	bool	Cửa đang mở trong mô phỏng
alarm	bool	Đã có tamper; chỉ reset hoàn tất xóa báo động
phase	normal / locked_out / resetting	Pha điều phối nghiệp vụ
admin_authorized	bool	Bằng chứng quyền quản trị được lưu cho reset đang thực hiện

Bảng 2.1. Biến trạng thái; nguồn: smartlock/model.py, State [3].

```
s_init = (False, 0, False, False, normal, False)
```

Tích miền có 192 tổ hợp, là cận trên biểu diễn. Chỉ những tổ hợp đạt được từ initial bằng guard/action mới thuộc S. Constructor chỉ kiểm miền và kiểu; không cấm trạng thái cửa mở trái phép, vì như vậy sẽ che lỗi v0 trước khi bộ kiểm chứng nhìn thấy.

2.2. Đầu vào và bề mặt tương tác

Chín event theo thứ tự cố định: `password_ok`, `password_bad`, `open_door`, `close_door`, `tamper`, `admin_reset`, `reset_complete`, `reset_retry`, `wait`. Hàm `step(state,event,version,admin_authenticated=...)` là điểm tiếp nhận input. Input từ môi trường gồm kết quả kiểm tra mật khẩu, phát hiện tamper và xác thực admin. `reset_complete` là event nội bộ do bên gọi cung cấp trong mô phỏng; `reset_retry` chỉ hợp lệ ở v0.

`authenticated` không cấp quyền admin. Input `admin_authenticated=True` phải được nêu tường minh khi gọi admin_reset; sau đó action mới đặt `admin_authorized=True`. Đây là giả định tin cậy của mô hình, chưa phải một cơ chế xác thực quản trị triển khai trên thiết bị.

2.3. Bảng guard/action và hai điểm khác nhau v0/v1

Ký hiệu: a=authenticated, f=failed_attempts, d=door_open, b=alarm, p=phase, u=admin_authorized. Các biến không nêu trong action giữ nguyên. Mỗi sự kiện hợp lệ thực hiện một action nguyên tử.

Event	Guard v1	Action
password_ok	p=normal, !b, !a, !d	a=True, f=0
password_bad	Như password_ok; f<3	f'=min(f+1,3); p'=locked_out khi f'=3
open_door	p=normal, !b, !d, a	d=True
close_door	p=normal, d	d=False, a=False
tamper	p khác resetting	b=True, d=False, a=False
admin_reset	p khác resetting; admin input=True	p=resetting, u=True, a=False, d=False
reset_complete	p=resetting	Trả đúng INITIAL_STATE
wait	p khác resetting	Giữ nguyên state
reset_retry	Không có guard v1	Từ chối; v0 có self-loop khi resetting

Bảng 2.2. Quy tắc thực tế; nguồn: model.py::step, dòng 63-123 [3].

Đúng lần sai thứ ba: code tính `attempts` mới rồi so với 3 để cập nhật phase. Một lần đúng hoặc reset hoàn tất xóa chuỗi sai; wait không xóa. P2 là điều kiện một chiều nên unit test lần thứ ba vẫn bắt buộc.

BUG-01 của v0 bỏ yêu cầu a trong guard mở cửa. BUG-02 thêm reset_retry tại mọi resetting. V1 sửa đúng hai điểm đó. Không thay đổi năm chức năng. Trong reset v1 chỉ có reset_complete: không thêm wait hoặc cạnh từ chối thành self-loop. Ngoài reset luôn có wait nên quan hệ vẫn total.

Sự kiện không biết hoặc guard sai phát sinh `InvalidEvent`. Nó không trả next_state và không tạo bước trong Kripke. Gọi sai bị từ chối khác với self-loop hợp lệ được mô hình hóa bằng một event rõ ràng.

2.4. Cấu trúc K và atomic propositions

Với mỗi phiên bản, `K=(S,S0,R,L)`: S là toàn bộ state do BFS sinh; S0={s0}; R gồm các cặp nguồn/đích của transition hợp lệ; L ánh xạ state sang tập atomic propositions (AP) đúng. Danh sách `(source,event,target)` được giữ riêng để mô phỏng và replay. Cùng nguồn/đích có thể có hai event.

AP	Điều kiện trên State
authenticated, door_open, alarm	Đọc trực tiếp trường bool tương ứng
locked_out	phase == 'locked_out'
attempts_eq_3	failed_attempts == 3
resetting	phase == 'resetting'
admin_authorized	Đọc trường admin_authorized độc lập với phase
idle	Toàn bộ sáu giá trị bằng INITIAL_STATE

Bảng 2.3. Cách tính nhãn trong model.py::labels [3].

`locked_out` không được định nghĩa bằng bộ đếm 3; `admin_authorized` không suy ra từ resetting. Nếu làm vậy, P2/P4 sẽ thành đúng do định nghĩa và không kiểm tra được lỗi cập nhật pha/quyền. Idle không chỉ là "không reset": alarm hoặc f=1 vẫn có thể ở phase normal nhưng chưa idle.

smartlock/model.py : dòng 48-60

```
48  def labels(state: State) -> frozenset[str]:
49      """L(s): AP được tính từ dữ liệu thật, không gán nhãn theo tên state."""
50      propositions = {
51          "authenticated": state.authenticated,
52          "door_open": state.door_open,
53          "alarm": state.alarm,
54          "locked_out": state.phase == "locked_out",
55          "attempts_eq_3": state.failed_attempts == 3,
56          "resetting": state.phase == "resetting",
57          "admin_authorized": state.admin_authorized,
58          "idle": state == INITIAL_STATE,
59      }
60      return frozenset(name for name, holds in propositions.items() if holds)
```

Lớp toolkit chỉ chứa graph; nhóm viết State, guard/action, BFS sinh state, adapter và các phép kiểm tra. `load_g2_model` dùng lại BFS rồi so toàn bộ snapshot; không nhận một JSON có labels/cạnh sai như bằng chứng đúng. Các sơ đồ đầy đủ ở Phụ lục A/B xuất từ chính K này.

2.5. BFS tự sinh state và quy mô thực tế

`smartlock/kripke.py::build_model(version)` bắt đầu bằng duy nhất INITIAL_STATE. Deque lưu state chờ mở rộng; dictionary state_ids nhận diện trùng qua cả sáu trường. `successors` gọi cùng `step` với simulator, chỉ yield những event có guard đúng. Không đọc danh sách state viết tay.

smartlock/kripke.py : dòng 164-179

```
164
165  while queue:
166      current = queue.popleft()
167      source = state_ids[current]
168      for event, next_state in successors(current, version):
169          if next_state not in state_ids:
170              target = f"s{len(states)}"
171              state_ids[next_state] = target
172              states[target] = next_state
173              graph.add_state(target, labels=labels(next_state))
174              distances[target] = distances[source] + 1
175              parents[target] = Transition(source, event, target)
176              queue.append(next_state)
177          target = state_ids[next_state]
178          transitions.append(Transition(source, event, target))
179          graph.add_transition(source, target)
```

Khi đích mới xuất hiện, code gán ID, nhãn, parent, depth rồi thêm cuối queue. Với mọi cạnh, kể cả về state cũ, code vẫn lưu transition có event và cập nhật R. FIFO bảo đảm lần khám phá đầu là đường ngắn nhất theo số bước. Không dùng các điều kiện an toàn làm bộ lọc BFS.

Ví dụ v1: s0 thử password_ok sinh s1, password_bad sinh s2; open_door bị từ chối; tamper sinh s3; admin_reset đã xác thực sinh s4. Từ s2, lần sai tiếp theo sinh s6; lần sai thứ ba từ s6 sinh s10 locked_out. Đường ba lần sai, tamper, admin_reset đạt độ sâu 5 ở s17.

Đại lượng	v0	v1
States reachable	21	18
Transitions có sự kiện	69	46
Cặp quan hệ R	65	42
D (độ sâu ngắn nhất lớn nhất)	5	5

Bảng 2.4. Số liệu đọc từ hai model.json và đối chiếu JSON G3/G4/G5.

V0 có 11 normal, 2 locked_out, 8 resetting; v1 có 8 normal, 2 locked_out, 8 resetting. Ba state cửa mở chưa xác thực f=0,1,2 bị loại ở v1 do sửa guard. Cả hai có 11 kiểm tra cấu trúc và 4 bất biến dữ liệu đạt; chúng không thay thế kiểm chứng CTL. Toàn bộ danh sách S/R/L nằm trong results/v0 và results/v1/model.json; không làm tròn số hay ép đủ 192 tổ hợp.

Chương 3. Đặc tả thuộc tính CTL

3.1. Ngữ nghĩa và phạm vi

CTL diễn tả các nhánh của cây thực thi. AG q nghĩa là q đúng ở mọi vị trí trên mọi đường; AF q nghĩa là mỗi đường phải có một vị trí hữu hạn thỏa q, kể cả vị trí hiện tại. Graph total nên đường có thể kéo dài vô hạn. Ngữ nghĩa lượng từ đường được đối chiếu tài liệu NuSMV [7].

Safety loại trừ điều xấu bằng phản ví dụ hữu hạn. Liveness yêu cầu tiến triển; vài bước chưa hoàn tất không đủ bác bỏ, cần một đường vô hạn không đạt đích. Code chỉ cài đúng năm công thức; FORMULAS là chuỗi mô tả, không đưa vào parser CTL tổng quát.

3.2. P1 - Cửa mở phải có quyền

```
AG(door_open -> (authenticated AND NOT locked_out))
```

Nghiệp vụ: cửa mở phải có phiên đã xác thực và tài khoản không bị khóa. Toán học: mọi state reachable có AP door_open thì có authenticated và không có locked_out. AP lấy từ door_open/authenticated và phase thật. Đây là safety; `bad_state` nhận diện cửa mở kèm thiếu xác thực hoặc locked_out. Một đường từ initial tới bad đủ bác bỏ AG.

P1 kiểm tra quyền tại trạng thái cửa mở, không chỉ thời điểm gọi mở. Vì thế tamper và close_door đồng thời đóng cửa khi hủy phiên. Kết luận dựa trên giả định action nguyên tử. V0 có ba bad state s3/s8/s13; v1 không có bad state. BMC trả phản ví dụ ngắn nhất ở k=1 cho v0.

3.3. P2 - Lockout đi cùng bộ đếm ba

```
AG(locked_out -> attempts_eq_3)
```

Nghiệp vụ: state khóa tài khoản phải ghi nhận ba lần sai. Toán học: mọi state reachable có phase locked_out phải có failed_attempts=3. Hai AP được tính độc lập nên có ý nghĩa kiểm tra thực sự. Bad là locked_out và không attempts_eq_3; đây là safety, cả hai phiên bản thỏa.

Công thức một chiều không đòi mọi state f=3 phải locked_out, vì reset có thể giữ f=3 tạm thời. Nó cũng không tự bảo đảm khóa ngay lần sai thứ ba. Các tests chạy từng lần 1,2,3 kiểm tra yêu cầu thời điểm này. Không thêm công thức CTL thứ sáu để thay đổi phạm vi đã thống nhất.

3.4. P3 - Báo động giữ cửa đóng

```
AG(alarm -> NOT door_open)
```

Nghiệp vụ: phát hiện tamper thì không cho cửa tiếp tục mở. Toán học: không có state reachable đồng thời mang AP alarm và door_open. Cả hai AP đọc trực tiếp hai bool độc lập. Đây là safety; bad là alarm AND door_open. CTL quét toàn bộ S, BMC tìm đường tới tập bad, đều cho kết quả thỏa ở v0/v1. Giả định tamper đóng cửa trong cùng một action, chưa mô hình hóa độ trễ cơ cấu vật lý.

3.5. P4 - Reset có bằng chứng admin

```
AG(resetting -> admin_authorized)
```

Nghiệp vụ: chỉ reset khi yêu cầu đã được xác thực quản trị. Toán học: mọi state reachable phase resetting phải có trường quyền True. Đây là safety; bad là resetting AND NOT admin_authorized. Hàm labels không tự gán quyền chỉ vì thấy phase resetting. Cả hai phiên bản thỏa.

Giả định quan trọng là input admin_authenticated đáng tin cậy và strict bool. P4 không chứng minh thuật toán xác thực admin thật; thuật toán đó không có trong chương trình mục tiêu. Tests xác nhận mật khẩu người dùng đúng vẫn không đủ gọi admin_reset nếu input quyền False.

3.6. P5 - Mọi reset đều phải hoàn tất

```
AG(resetting -> AF idle)
```

Nghiệp vụ: ở mọi thời điểm reset bắt đầu/đang thực hiện, mọi cách tiếp diễn hợp lệ phải trở lại trạng thái sạch sau hữu hạn bước. AP resetting đọc phase; idle yêu cầu toàn bộ State bằng INITIAL_STATE. Đây là liveness, không có một trạng thái resetting đơn lẻ nào tự đủ làm bad state.

V0 có nhánh reset_complete nhưng cũng có reset_retry tự lặp. Tồn tại đường hoàn tất chỉ phù hợp lượng từ tồn tại; AF đòi mọi đường. V1 bỏ self-loop trong resetting, chỉ giữ complete về s0. Không thêm fairness để bỏ qua nhánh xấu v0. Không suy ra thời hạn reset theo giây: simulator chỉ có bước rời rạc và bên gọi phải cung cấp event nội bộ.

Các tiền đề đều đạt được: số state kích hoạt P1..P5 lần lượt v0 là 4,2,8,8,8 và v1 là 1,2,8,8,8. Vì vậy kết luận HOLDS không chỉ do tiền đề không bao giờ xuất hiện. Các tập AP chồng nhau, không cộng thành |S|.

3.7. P5 dùng CTL least fixed-point

Đặt Succ(s) là tập kế tiếp trong R. Tập thỏa AF idle được tính từ:

```
Z0 = {s | idle(s)}

Z(n+1) = Z(n) union {s | Succ(s) subset Z(n)}
```

Đây là điểm bất động nhỏ nhất: idle đúng ngay; nếu mọi kế tiếp chắc chắn tới idle thì state hiện tại cũng vậy. Totality được kiểm tra trước để không chấp nhận deadlock do điều kiện tập rỗng. Phải dùng "mọi kế tiếp", không dùng "có một kế tiếp".

smartlock/ctl.py : dòng 95-109

```
 95 def af_idle_fixed_point(model: SmartLockModel) -> tuple[set[str], list[list[str]]]:
 96     """μZ.(idle ∪ AX Z), trên quan hệ total; trả tập AF idle và các vòng lặp."""
 97     if any(not model.graph.transitions.get(sid) for sid in model.states):
 98         raise ValueError("AF yêu cầu Kripke total, không chấp nhận deadlock")
 99     order = list(model.states)
100     reached = {sid for sid, state in model.states.items() if "idle" in labels(state)}
101     history = [[sid for sid in order if sid in reached]]
102     while True:
103         expanded = reached | {
104             sid for sid in order if model.graph.transitions[sid] <= reached
105         }
106         if expanded == reached:
107             return reached, history
108         reached = expanded
109         history.append([sid for sid in order if sid in reached])
```

V0: Z chỉ có s0, lịch sử kích thước [1]. Mỗi resetting có kế tiếp là chính nó nên không thể được thêm. V1: vòng đầu thêm tám resetting vì mọi kế tiếp của chúng là s0; lịch sử [1,9], sau đó ổn định. Tám state resetting v1 thuộc AF idle; các state normal có wait không nhất thiết thuộc AF idle nhưng không làm P5 sai.

Hàm `check_ctl` yêu cầu mọi resetting reachable nằm trong Z. Nếu không, `_liveness_trace` lấy prefix BFS tới một resetting ngoài Z, theo các cạnh vẫn ngoài Z tới khi gặp ID đã thấy. Đồ thị hữu hạn bảo đảm có state lặp. Prefix cộng chu trình khép kín tạo lasso, mô tả đường vô hạn không có idle kể từ khi reset bắt đầu.

`validate_counterexample` replay từng event, kiểm loop_start/reset_step, chu trình không rỗng, đầu/cuối trùng nhau, không đi qua idle sau reset. Idle trong prefix trước reset được phép. Phép kiểm này tránh dùng một đường đã hoàn tất reset rồi reset lại để kết luận sai về một reset cũ.

Chương 4. Kiểm chứng bằng BMC và Z3

4.1. Mã hóa initial, miền và quan hệ chuyển tiếp

BMC xét các đường dài k bước bằng công thức ràng buộc. Có k+1 biến nguyên x0..xk mã hóa ID của chính state G2, và k biến e0..e(k-1) mã hóa event. Không tạo một bộ state nghiệp vụ khác trong SMT. Với k=0 vẫn kiểm tra initial và bad nhưng chưa có transition.

```
I(x0) AND domains AND T(x0,e0,x1) AND ...

AND T(x(k-1),e(k-1),xk) AND OR(i=0..k) bad_j(xi)
```

I đặt x0=id(s0). Mỗi xi thuộc [0,|S|-1], mỗi ei thuộc [0,8]. T là OR của đúng các bộ ba G2, mỗi nhánh là một AND nguồn/event/đích. Hai event cùng đầu/cuối vẫn được mã hóa riêng. Không đưa thuộc tính cần chứng minh vào T như một giả định đúng, vì như vậy sẽ loại sẵn lỗi cần tìm.

smartlock/bmc.py : dòng 10-17

```python
def transition_constraint(model: SmartLockModel, source, event, target):
    """T(x,e,y): OR đúng các cạnh source/event/target đã sinh ở G2."""
    ids = {sid: i for i, sid in enumerate(model.states)}
    event_ids = {name: i for i, name in enumerate(EVENTS)}
    return Or([
        And(source == ids[edge.source], event == event_ids[edge.event], target == ids[edge.target])
        for edge in model.transitions
    ])
```

smartlock/bmc.py : dòng 20-32

```python
def encode_path(model: SmartLockModel, bound: int) -> tuple[list, list, list]:
    """Unroll k bước: k+1 biến state ID, k biến event, I/domain/T."""
    if type(bound) is not int or bound < 0:
        raise ValueError("bound phải là số nguyên không âm")
    states = [Int(f"x_{i}") for i in range(bound + 1)]
    events = [Int(f"e_{i}") for i in range(bound)]
    ids = {sid: i for i, sid in enumerate(model.states)}
    constraints = [Or([states[0] == ids[sid] for sid in model.graph.initial])]
    constraints += [And(state >= 0, state < len(ids)) for state in states]
    for i, event in enumerate(events):
        constraints.append(And(event >= 0, event < len(EVENTS)))
        constraints.append(transition_constraint(model, states[i], event, states[i + 1]))
    return states, events, constraints
```

`encode_path` chỉ tạo biến và danh sách constraints; `check_bound` mới gửi chúng cho Z3. Bad được tính bằng AP thật từ State: P1 là cửa mở kèm thiếu quyền; P2 là khóa sai bộ đếm; P3 là báo động/cửa mở; P4 là reset thiếu quyền. P5 bị API BMC bad-state từ chối để tránh dùng sai phương pháp. Đây là BMC trên graph đã BFS đầy đủ, không phải engine symbolic tự trích mô hình từ mọi chương trình Python.

4.2. Lời gọi Z3 thật và trạng thái kết quả

Trong `smartlock/bmc.py::check_bound`, code tạo Solver, đặt timeout 10.000 ms, thêm constraints, thêm bad, lưu SMT-LIB rồi gọi check thật. Nếu tập bad rỗng vẫn thêm BoolVal(False) và gọi solver, không tự gán UNSAT khi chưa thực thi. API add/check/model được dùng theo Z3Py [6].

smartlock/bmc.py : dòng 51-63

```
51  ids = list(model.states)
52  bad_ids = [i for i, sid in enumerate(ids) if bad_state(model.states[sid], property_id)]
53  solver = Solver()
54  solver.set(timeout=10000)
55  solver.add(*constraints)
56  bad = Or([state == bad_id for state in states for bad_id in bad_ids]) if bad_ids else BoolVal(False)
57  solver.add(bad)  # Tập bad rỗng vẫn gọi solver, SMT-LIB ghi rõ (assert false).
58  result["smt2"] = solver.to_smt2()
59  result["solver_called"] = True
60  outcome = solver.check()
61  result["solver_status"] = str(outcome).upper()
62  if outcome == sat:
63      witness = solver.model()
```

Kết quả	Ý nghĩa trong chương trình
SAT	Có phép gán biểu diễn đường vi phạm trong bound; phải giải mã và replay
UNSAT	Không có đường vi phạm trong công thức với bound đang xét
UNKNOWN	Solver chưa kết luận; lưu reason_unknown và dừng tăng bound
ERROR	Lỗi ứng dụng; tách khỏi solver_status, không phải phản hồi thứ tư của Z3

Bảng 4.1. Phân biệt phản hồi solver với trạng thái ứng dụng.

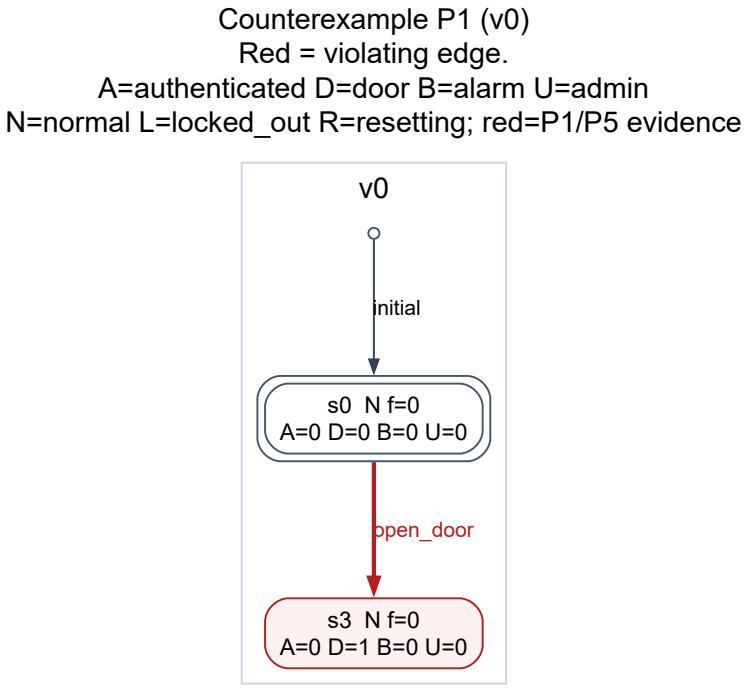
`solver_called`, `solver_status` và `status` được lưu riêng. Nếu SAT nhưng giải mã/replay lỗi, solver_status vẫn SAT còn status là ERROR. Không được công bố counterexample hợp lệ. Nếu chưa check thì solver_status=None. UNKNOWN/ERROR trong tests là giả lập có chủ đích; lần kiểm chứng chính thức không gặp chúng.

Phiên bản	Lượt check	SAT	UNSAT	UNKNOWN
v0	20	1	19	0
v1	24	0	24	0

Bảng 4.2. 44 query G3: 1 SAT, 43 UNSAT, 0 UNKNOWN, 0 lỗi ứng dụng; không cộng lời gọi trong tests. Header SMT-LIB không thay solver_status.

4.3. Phản ví dụ P1 v0 và cách replay

Ở v0, thứ tự EVENTS gán open_door mã 2; s0 có mã 0 và s3 mã 3. Bad P1 gồm s3/s8/s13. Với k=0, x0=0 không thuộc bad nên Z3 trả UNSAT. Với k=1 có phép gán x0=0, e0=2, x1=3 nên Z3 trả SAT.



Hình 4.1. Counterexample P1 từ G3, vẽ từ cạnh thật G2; đỏ là bước vi phạm.

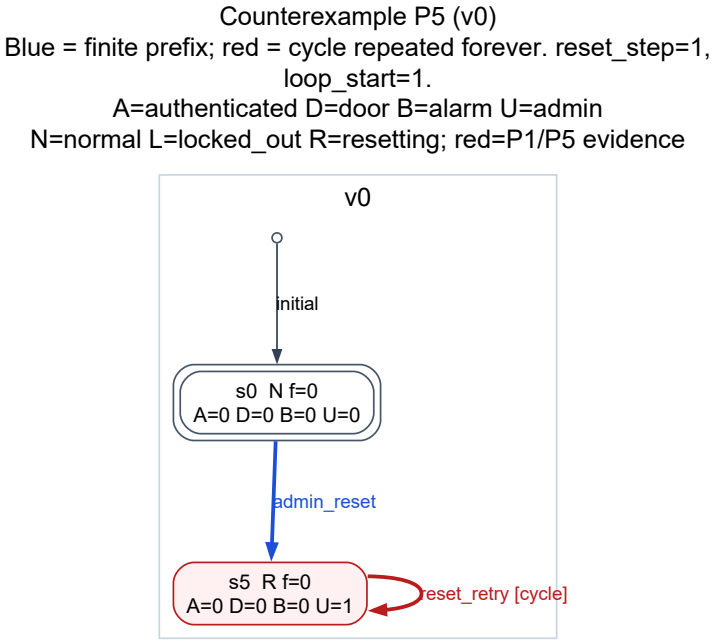
```
s0 --open_door--> s3

s3 = (False, 0, True, False, normal, False)
```

Cửa mở nhưng authenticated=False làm P1 sai. Do mọi bound nhỏ hơn đã thật sự UNSAT, đây là phản ví dụ ngắn nhất theo số transition. `check_safety_bmc` thử k từ 0, không nhảy thẳng tới một bound lớn.

Sau SAT, `solver.model()` cho giá trị các Int. Code kiểm miền, đổi ngược thành ID/event, dùng `make_trace` thêm sáu giá trị và labels, đánh dấu violation_step. `validate_counterexample` gọi `replay_trace`: initial đúng, từng cạnh tồn tại, admin input đúng, gọi lại step G2 cho đúng state đích; sau đó kiểm đúng bad đầu tiên. Trace sửa event/label hoặc thêm cạnh không tồn tại sẽ không vượt qua replay.

4.4. Lasso P5 và completeness threshold



Hình 4.2. Prefix xanh: s0 --admin_reset [True]--> s5. Cycle đỏ: s5 --reset_retry--> s5. loop_start=reset_step=1; nguồn: G3/G4.

Từ s5 có thể chọn reset_retry mãi mà không gặp idle. Một chu trình không rỗng đã khép kín đủ biểu diễn đường vô hạn, không cần chạy test vô hạn. P5 do CTL fixed-point kết luận; không ghi SAT/UNSAT Z3 cho P5.

`reachable_depths` tính lại khoảng cách ngắn nhất trên R. Cả hai phiên bản có D=max_s dist(s0,s)=5. Nếu bad state đạt được, có đường ngắn nhất tới nó dài không quá D. Graph total cho phép kéo dài tiền tố ngắn hơn thành D bước, còn BMC kiểm bad ở mọi vị trí 0..D. Do đó UNSAT tới D đủ cho bốn safety dạng trạng thái hiện tại trên graph này.

D là ngưỡng đủ, không nhất thiết nhỏ nhất cho từng P. Khi k<D mà chưa có lỗi, BMC trả BOUNDED_NO_COUNTEREXAMPLE, holds=None; không được nói đã chứng minh toàn cục. UNKNOWN cũng không phải UNSAT. D không xử lý chu trình liveness: P5 vẫn cần fixed-point/lasso trên toàn graph.

Chương 5. Sinh mã và kết quả thực nghiệm

5.1. Sinh Python từ dữ liệu mô hình

`smartlock/artifacts.py::generate_python` đọc snapshot G2, kiểm miền, initial, endpoint và tính xác định của từng cặp state/event. Sau đó tuần tự hóa STATES, LABELS, INITIAL_STATE, EVENT_INPUTS, TRANSITIONS thành literal Python và nối bộ tra bảng RUNTIME. Không viết lại guard của mật khẩu/cửa/alarm; không viết tay một Smart Lock thứ hai.

smartlock/artifacts.py : dòng 88-96

```
88  table = {}
89  for edge in data["transitions"]:
90      pair = edge["source"], edge["event"]
91      target = edge["target"]
92      if pair[0] not in states or target not in states or pair[1] not in EVENTS:
93          raise ValueError(f"Cạnh có state/event không hợp lệ: {edge}")
94      if pair in table and table[pair] != target:
95          raise ValueError(f"Nondeterministic transition {pair}: {table[pair]} / {target}")
96      table[pair] = target
```

Nếu cùng source/event có hai target khác nhau, generator phát sinh ValueError trước khi ghi đầu ra. Hai file sinh chỉ import argparse/json, chạy không cần model.py, toolkit, Z3 hoặc JSON ngoài. Input step là ID state và event, output là ID đích; state_values trả đủ sáu giá trị.

generated/v1/smart_lock.py : dòng 119-132

```
119 def step(state, event, *, admin_authenticated=False):
120     if type(admin_authenticated) is not bool:
121         raise ValueError("admin_authenticated must be bool")
122     state_values(state)
123     if type(event) is not str:
124         raise InvalidEvent(f"Unknown event: {event!r}")
125     required = EVENT_INPUTS.get(event, {})
126     if required.get("admin_authenticated", False) and not admin_authenticated:
127         raise InvalidEvent("Admin verification is required")
128     try:
129         return TRANSITIONS[(state, event)]
130     except KeyError:
131         raise InvalidEvent(f"Event {event!r} is not enabled at {state}") from None
```

Input sai/guard không hợp lệ phát sinh InvalidEvent, không thành cạnh chờ. Metadata yêu cầu admin=True được giữ trong mã sinh. V0 bảo toàn cả cạnh mở cửa trái phép và self-loop reset_retry; v1 không có chúng. Test cô lập copy duy nhất file Python sang thư mục rỗng, chạy -I -S thành công. Hash snapshot nhúng trong file cho phép truy nguyên nguồn.

5.2. Đối chiếu toàn miền và kết quả kiểm chứng

`audit_generated` so initial, mọi state/label và toàn bộ bảng cạnh; sau đó lặp S × chín event × hai giá trị quyền admin. Kỳ vọng lấy từ step G2, kết quả lấy từ step sinh; so cả chấp nhận lẫn từ chối. Tập transition thực hiện phải bằng đúng tập G2, không chỉ là tập con.

Phiên bản	Phép so	Chấp nhận	Từ chối	Cạnh khớp
v0	378	125	253	69
v1	324	82	242	46

Bảng 5.1. 702 phép so, 207 chấp nhận đúng, 495 từ chối đúng; 115/115 cạnh phân biệt khớp. Số chấp nhận lớn hơn số cạnh do hai input quyền.

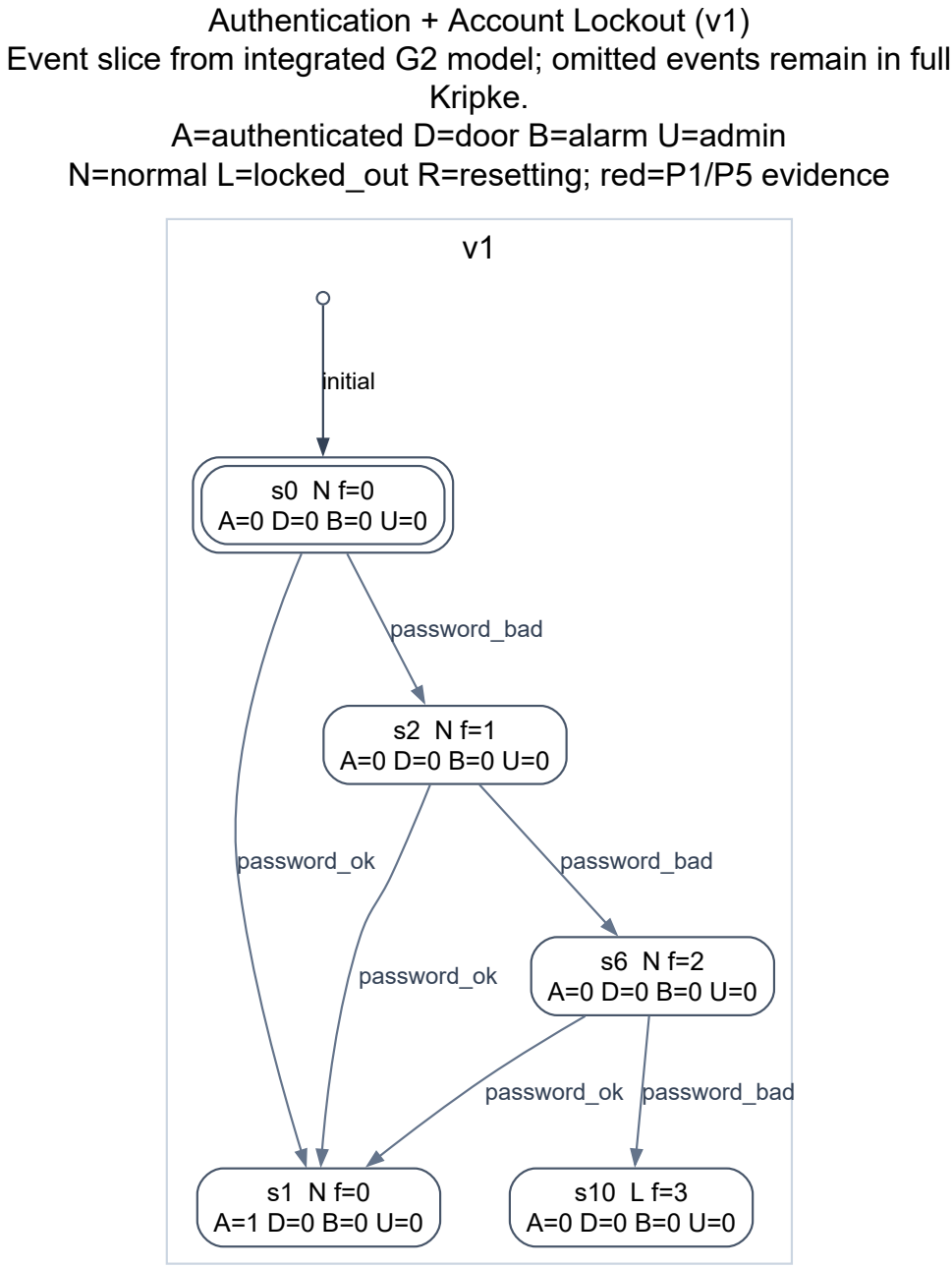
P	Phương pháp	v0	v1	Bound safety
P1	Đồ thị + Z3	VIOLATED	HOLDS	v0: 0..1
P2	Đồ thị + Z3	HOLDS	HOLDS	0..5
P3	Đồ thị + Z3	HOLDS	HOLDS	0..5
P4	Đồ thị + Z3	HOLDS	HOLDS	0..5
P5	CTL fixed-point	VIOLATED	HOLDS	Không áp dụng

Bảng 5.2. Kết quả G3. Riêng P1 v1 chạy k=0..5; P1 v0 dừng ở k=1. P2-P4 cả hai chạy 0..5. P5 không dùng bound safety; CE tại Hình 4.2.

G5 chạy lại 199 tests dự án (78 G2 + 72 G3 + 49 G4) và 4 tests tham chiếu Python G1: **203 PASS, 0 FAIL**. 11 kịch bản độc lập G4 đạt kết quả mong đợi; exit 1 ở input bị từ chối là kết quả đúng. Test v0 PASS nghĩa hành vi lỗi được tái hiện, không có nghĩa v0 an toàn.

Nguồn số liệu: results/g5/evidence_audit.json, pytest.log, g1_reference_pytest.log; G2 model.json; G3 verification.json; G4 experiment_summary.json/CSV và standalone_examples.log. Không phát hiện khác biệt số liệu giữa JSON, CSV và log đã nghiệm thu.

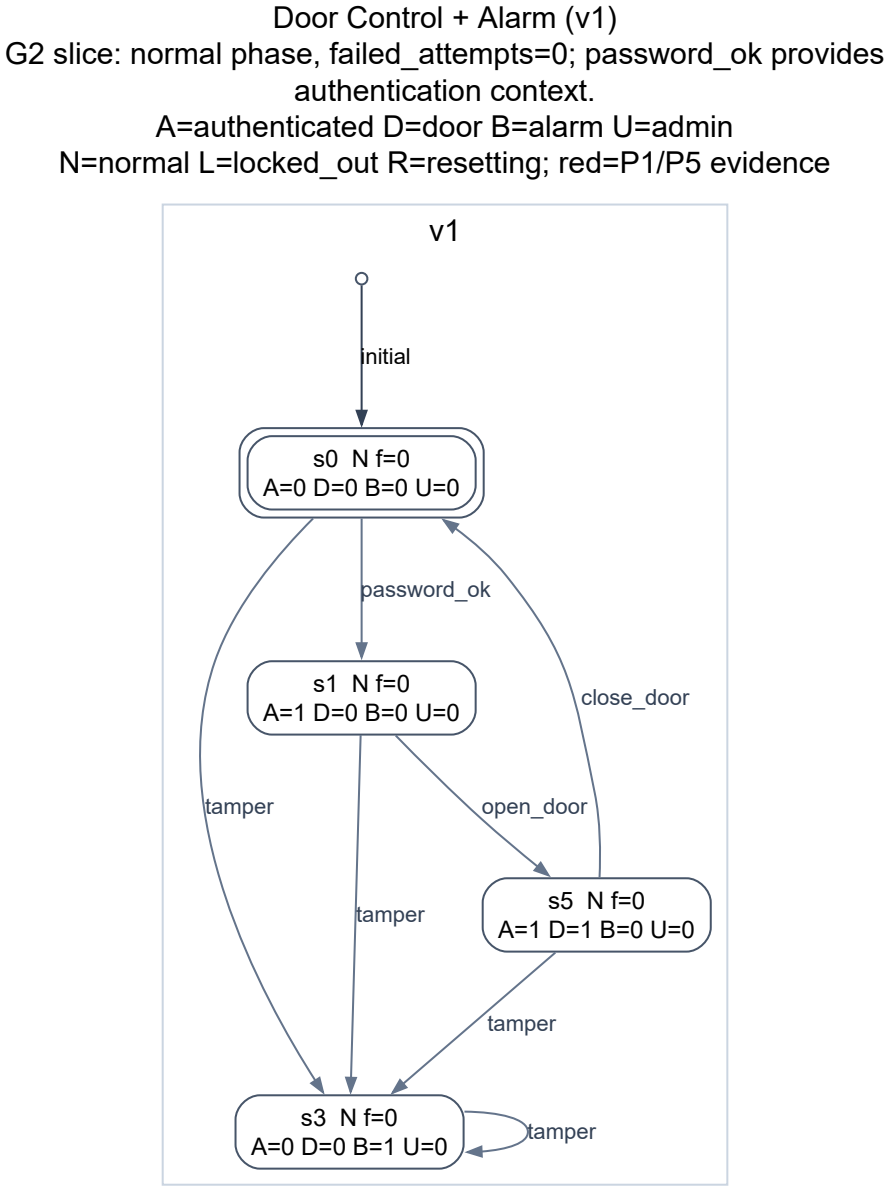
5.3. Authentication và Account Lockout



Hình 5.1. Lát cắt event mật khẩu từ mô hình tích hợp v1; f là bộ đếm, A là xác thực, D là cửa, B là báo động, U là quyền admin; N/L là pha.

Ba cạnh password_bad từ s0 qua s2, s6 tới s10 khóa đúng lần thứ ba. Password_ok từ f=1 hoặc f=2 về s1 và xóa bộ đếm. Các event còn lại không hiển thị trong lát cắt nhưng vẫn nằm trong K đầy đủ. Không dùng hình rút gọn này làm đầu vào để kết luận CTL.

5.4. Door Control, Alarm và các sơ đồ còn lại



Hình 5.2. Lát cắt v1: phase normal, f=0, các event mật khẩu đúng/cửa/tamper.

Mở cửa đi qua s1 đã xác thực. Tamper từ s0/s1/s5 tới s3 có alarm=True và cửa đóng; tamper tại s3 tự lặp. Admin Recovery được minh họa ở Phụ lục C cho cả hai phiên bản. Hai graph đầy đủ ở Phụ lục A/B giữ đủ 21/18 node và 69/46 event edge; hai phản ví dụ ở Hình 4.1/4.2. Tổng bảy sơ đồ cùng nguồn tích hợp, mỗi hình có DOT/SVG/PNG trong results/g4.

BUG-01 là bỏ xác thực trước thao tác nhạy cảm, tương ứng ý tưởng CWE-306 [9]; đây là ánh xạ phân tích, không phải kết quả quét CWE. BUG-02 là lỗi thiết kế tiến triển của quy trình reset. V1 bổ sung guard authenticated và bỏ nhánh retry trong reset. Không có lỗi ngoài dự kiến phải chỉnh mô hình để ép kết quả phù hợp dự đoán.

Chương 6. Thảo luận và hạn chế

6.1. Ý nghĩa của bằng chứng

CTL và BMC safety đồng ý trên cùng graph. CTL quét toàn bộ reachable states; Z3 tìm witness theo ràng buộc và bound; replay kiểm lại bằng guard/action gốc. Hai cách kiểm tra cùng dữ liệu giúp phát hiện nhầm lẫn giữa đặc tả, encoding và giải mã, nhưng không tự làm một đặc tả sai trở thành đúng. Các tests cố tình làm sai labels, trace hoặc cạnh nhằm kiểm tra khả năng từ chối bằng chứng không hợp lệ.

Audit toàn miền của mã sinh xác nhận quan hệ một bước trùng G2. Từ initial trùng nhau, quy nạp theo số bước cho các chuỗi input hợp lệ cho kết quả state trùng nhau. Điều này bảo toàn hành vi của simulator bảng hữu hạn; không phải chứng minh tương đương với firmware bất kỳ.

6.2. Giới hạn của mô hình và phương pháp

Mật khẩu đúng/sai và quyền admin là kết quả trừu tượng. Chưa phân tích lưu trữ mật khẩu, chống đoán tự động theo thời gian, kênh giao tiếp hoặc xác thực mật mã. Không xét mất điện, kẹt chốt, lỗi cảm biến, tiến trình bị dừng, nhiều người dùng đồng thời hoặc race condition.

Action nguyên tử giúp state không chứa cửa còn mở trong khi tamper đang đóng dở. P5 giả định ưu tiên reset nội bộ: ở resetting v1 chỉ có complete. Simulator không có đồng hồ/scheduler, bên gọi phải gửi reset_complete. Dừng gửi event hoặc lặp gọi sai rồi bỏ qua exception không được tự tính là một self-loop hợp lệ của K.

BMC ở k bất kỳ chỉ kết luận trong bound; kết luận toàn cục cho P1-P4 cần graph đầy đủ, encoding đúng và ngưỡng D=5 đã tính. D này không áp dụng máy móc cho P5. BMC trên graph đã BFS không giải quyết state explosion và không phải công cụ kiểm chứng tổng quát cho Python.

6.3. Phạm vi kết luận và hướng cải thiện

V1 thỏa đúng năm CTL trên mô hình hữu hạn đã nêu, không chứng minh thiết bị thực an toàn tuyệt đối. P2 vẫn cần test đúng lần sai thứ ba; không suy ngược công thức một chiều. Nếu triển khai thực tế cần đặc tả thêm thời gian, lỗi phần cứng và cơ chế xác thực. Những hướng đó chỉ là thảo luận, không được triển khai thêm trong bài tập lớn này.

Chương 7. Kết luận và tài liệu tham khảo

Đề tài đã nối đặc tả nghiệp vụ với mô hình tích hợp, CTL, BMC/Z3 và simulator sinh tự động. Hai lỗi cố ý minh họa khác biệt safety/liveness: P1 có phản ví dụ hữu hạn ngắn nhất; P5 có đường vô hạn qua chu trình. V1 loại hai nhánh lỗi, thỏa năm CTL; mã sinh khớp toàn bộ transition. Giá trị chính là kết luận có thể truy nguyên và chạy lại, cùng giới hạn được nêu rõ, thay vì khẳng định an toàn dựa trên vài ví dụ.

7.1. Tài liệu tham khảo và nguồn mã

[1] Vũ Quang Dũng. Hướng dẫn Bài tập lớn: Chủ đề, phạm vi và quy trình thực hiện. CSE703093, Phenikaa; references/BTL_01_HuongDan_DeTai.pdf.

[2] Vũ Quang Dũng. Hướng dẫn Báo cáo Bài tập lớn. CSE703093; references/BTL_02_HuongDan_BaoCao.pdf; định dạng tr. 4-6, code tr. 20, checklist tr. 22, AI tr. 23.

[3] Mã Smart Lock G1-G4 trong gói nộp: smartlock/, generated/, run.py, tests/; tài liệu thiết kế và giải thích docs/G1-G4. Trích code tự động từ file thật; số dòng là số dòng phiên bản nghiệm thu G4.

[4] softsec-toolkit, bộ mã giảng viên cung cấp. Tái sử dụng nguyên lớp toolkit/softsec-toolkit/ch1_models/kripke.py; tham khảo ví dụ SMT/BMC ch3_verify/02_smt_examples.py và 03_bmc_manual.py; không sửa toolkit.

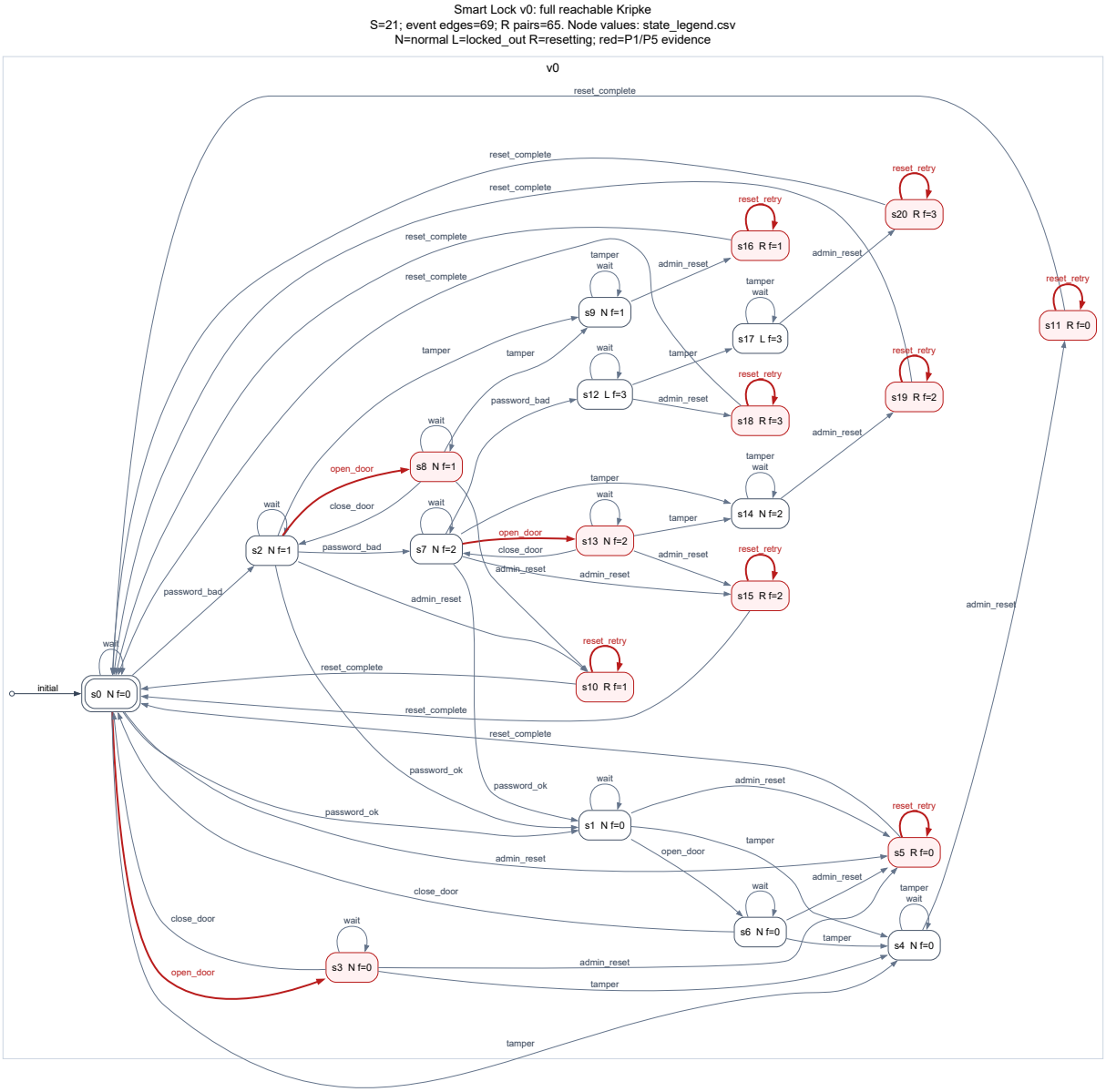
[5] Slide học phần: Chuong1_Phan2_ATPM.pdf, phần Kripke/CTL; Chuong3_Phan1_ATPM.pdf và Chuong3_Phan2_ATPM.pdf, đặc tả và BMC. Lab1/Lab6 dùng làm tham chiếu; log chạy riêng ở results/g1 và results/g5.

[6] Microsoft/Z3. Z3 API in Python, mục Solvers. [7] NuSMV. Temporal Logic. [8] Graphviz. SVG output. [9] MITRE. CWE-306. Các trang trực tuyến được đối chiếu ngày 15/10/2026.

7.2. Công khai hỗ trợ AI

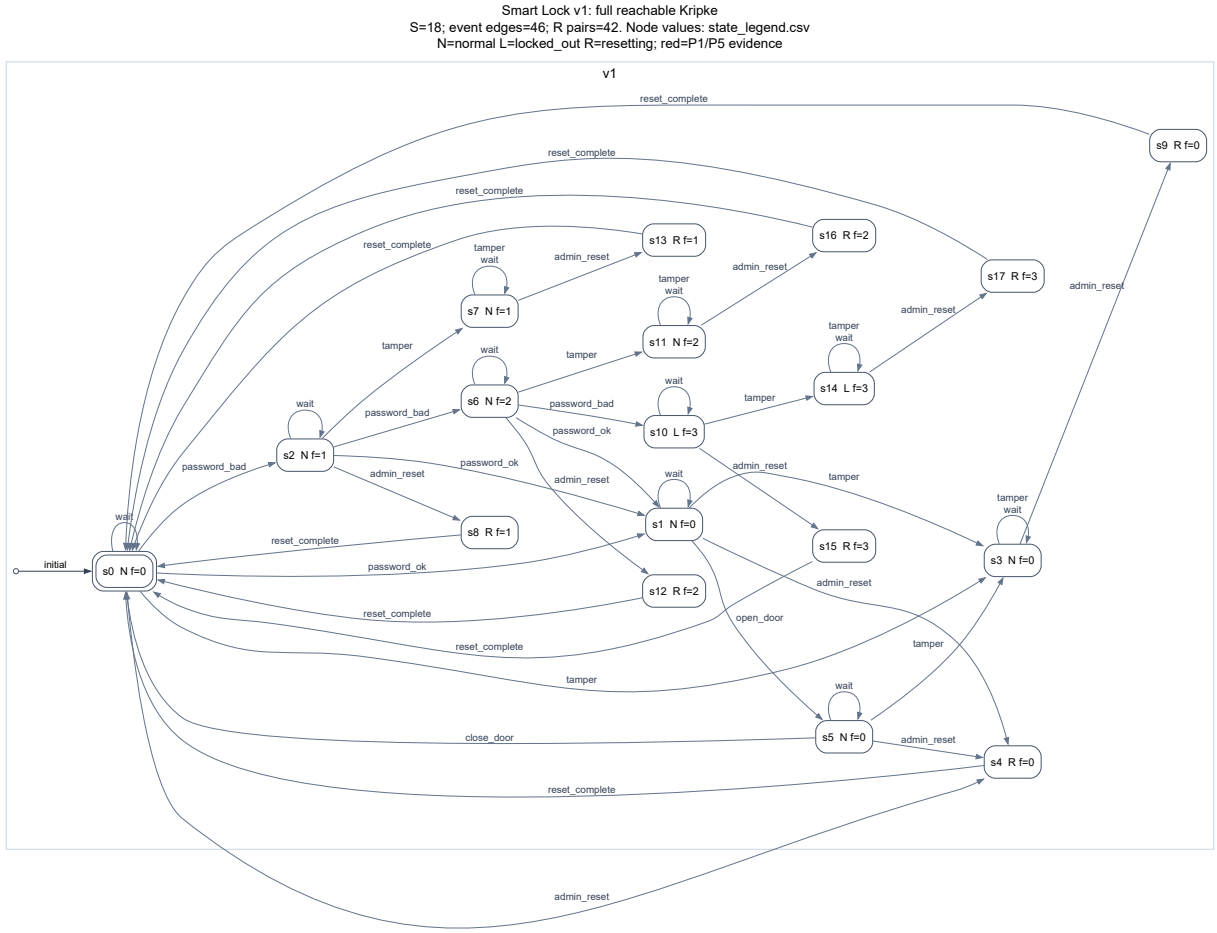
Codex/ChatGPT hỗ trợ thiết kế, viết phần mã bổ sung G1-G4, tests, tài liệu tiếng Việt, script biên soạn và rà soát báo cáo G5. Đây là mức hỗ trợ đáng kể, không chỉ sửa chính tả. Công cụ đã chạy lệnh/tests và thu log trên workspace; nhóm phải đọc, kiểm tra, chịu trách nhiệm và điền phân công thực tế trước khi nộp. Không tuyên bố bốn sinh viên tự viết toàn bộ mã; khai báo này thực hiện yêu cầu liêm chính [2, tr. 23].

Phụ lục A. Kripke tổng thể v0



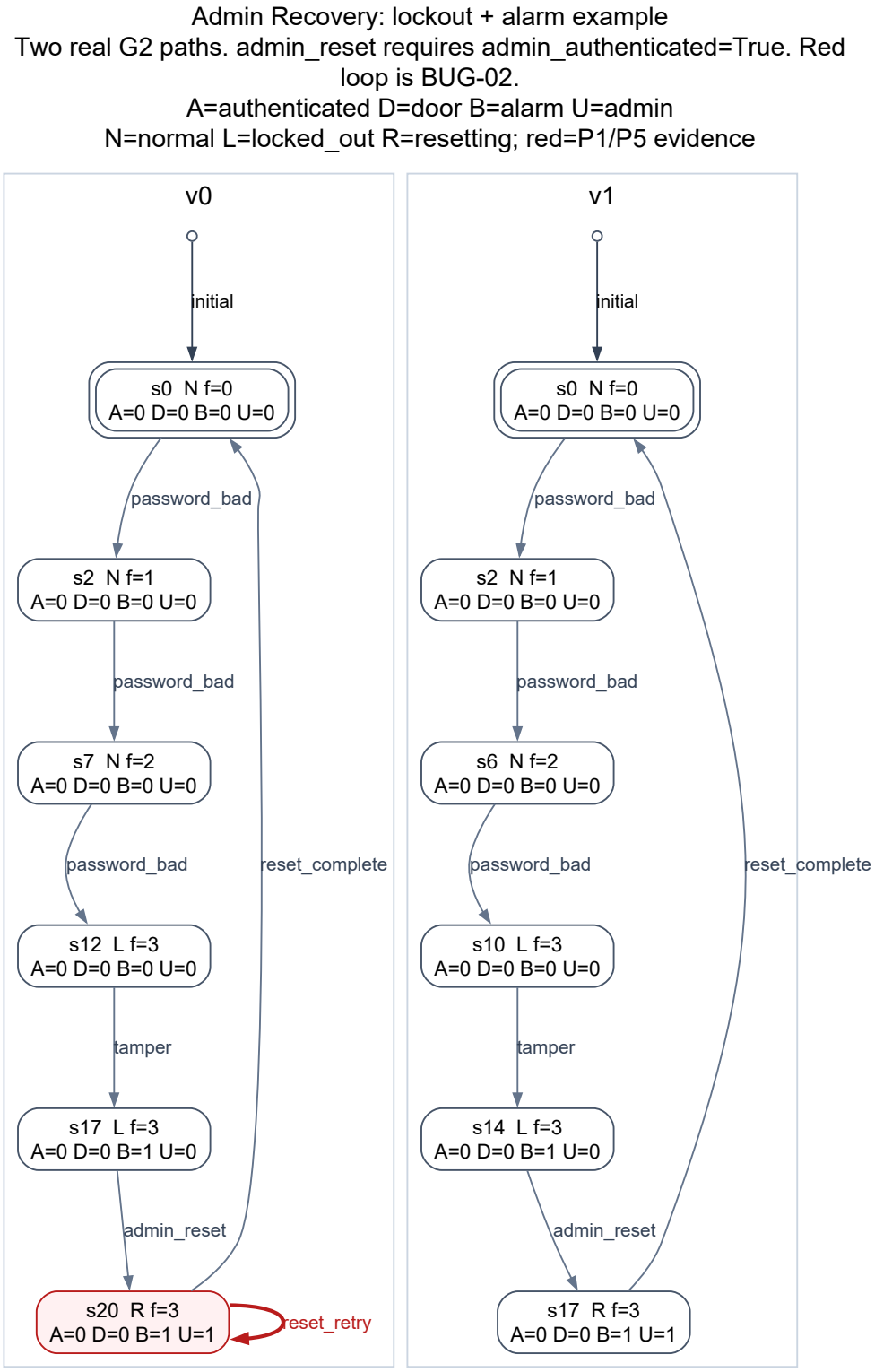
Hình A.1. Toàn bộ K v0: 21 states, 69 transition gắn event, 65 cặp R. Đỏ đánh dấu bằng chứng P1/P5; initial có hai viền. N/L/R là phase, f là failed_attempts. Nhiều event cùng nguồn/đích được gộp nhãn trên một mũi tên; không bỏ event. Hình vector có thể phóng lớn; mở SVG gốc ở results/g4/diagrams/kripke_v0.svg để xem chi tiết. Sáu giá trị/AP của từng ID được tra trong results/g4/state_legend.csv.

Phụ lục B. Kripke tổng thể v1



Hình B.1. Toàn bộ K v1: 18 states, 46 transition gắn event, 42 cặp R. Không còn state mở cửa chưa xác thực và reset_retry trong resetting. Giữ các self-loop wait ngoài reset. ID chỉ so được trong cùng version; ví dụ s5 v1 là cửa mở hợp lệ, s5 v0 lại là resetting. Dùng bảng state CSV để đọc đủ giá trị, và SVG gốc khi cần phóng lớn nhãn cạnh.

Phụ lục C. Admin Recovery



Hình C.1. Hai đường thật: ba lần sai → lockout → tamper → admin_reset có quyền → reset_complete. V0 còn self-loop đỏ reset_retry; v1 chỉ cho complete. Đây là ví dụ lấy từ graph tích hợp, không phải một mô hình độc lập. A/D/B/U và N/L/R theo chú giải ở Hình 5.1.

Phụ lục D. Lệnh chạy và chỉ mục bằng chứng

Chạy từ thư mục gốc gói nộp. Thay `python` bằng interpreter đã cài Python 3.10 trở lên. Để tái sinh ảnh và chạy tests G4 cần Graphviz dot trong PATH hoặc GRAPHVIZ_DOT. Không cần dot để chỉ chạy simulator sinh.

```
python -m pip install -r requirements.txt
python -X utf8 -m pytest --color=no -p no:cacheprovider -v
python -X utf8 run.py --output-dir tmp/recheck
python -X utf8 run.py --verify --output-dir tmp/recheck
python -X utf8 run.py --artifacts --output-dir tmp/recheck --generated-dir tmp/gen
```

Các lệnh tái lập vào tmp/recheck, mã sinh vào tmp/gen để giữ nguyên bằng chứng và hai file sinh đã bàn giao. Lệnh sau chạy độc lập và tái hiện lỗi:

```
python -I -S generated/v0/smart_lock.py --events open_door
python -I -S generated/v1/smart_lock.py --events open_door
python -I -S generated/v1/smart_lock.py --events password_ok open_door
```

Lệnh thứ hai trả exit 1 do bị từ chối đúng. Các lệnh Python tham chiếu G1 và lệnh xuất lại PDF được ghi đầy đủ trong README. Nguồn Markdown và build_pdfs.py nằm ở report/; font body Times New Roman 12pt, leading 18pt. Bảng/code dùng cỡ nhỏ hơn để giữ bố cục và số dòng nguồn.

Cần đối chiếu	Đường dẫn trong gói
States, events, R, AP, độ sâu	results/v0/model.json; results/v1/model.json
Từng query Z3, SMT2, trace	results/v0/verification.json; results/v1/verification.json
702 phép so, 115 cạnh, stdout	results/g4/experiment_summary.json; standalone_examples.log
Hash và số liệu đối chiếu G5	results/g5/evidence_audit.json; final_checks.json
Tests cuối cùng và ảnh PDF	results/g5/pytest.log; g1_reference_pytest.log; pdf_qa.json

Bảng D.1. Chỉ mục bằng chứng. Gói nộp không chứa môi trường ảo/cache; nhóm điền thông tin bìa và đổi tên thư mục/file theo quy định lớp.